بخش دوم: جریان داده ها در کافکا

فصل دوم: آغاز به کار با stream ها در کافکا

Kafka Streams

Kafka Streams یک کتابخانه سبک اما قدرتمند جاوا است که برای **غنی‌سازی، تبدیل و پردازش جریان‌های داده‌ای در زمان واقعی**طراحی شده است. در این فصل، یک معرفی کلی از Kafka Streams خواهید داشت. می‌توان این فصل را مانند یک «قرار اول» تصور کرد: کمی درباره پس‌زمینه Kafka Streams می‌آموزید و نگاهی اجمالی به ویژگی‌های آن می‌اندازید.

در پایان این فصل، شما با موارد زیر آشنا خواهید شد:

- جایگاه Kafka Streams در **اکوسیستمKafka**
- دلیل ساختKafka Streams
- ویژگی‌ها و خصوصیات عملیاتی این کتابخانه
- گروه هدف استفاده‌کنندگانKafka Streams
- مقایسه Kafka Streams با دیگر راهکارهای پردازش جریان داده
- نحوه ایجاد و اجرای یک برنامه سادهKafka Streams

اکوسیستمKafka

Kafka Streamsبخشی از مجموعه فناوری‌هایی است که به صورت جمعی **اکوسیستم Kafka** نامیده می‌شوند. در فصل ۱ یاد گرفتیم که هسته Apache Kafka شامل یک **Logتوزیع‌شده و فقط الحاقی (append-only)** است که می‌توان پیام‌ها را در آن تولید (produce) و مصرف (consume) کرد.

علاوه بر این، هسته Kafka شاملAPI های مهمی برای تعامل با این Log است که به دسته‌هایی از پیام‌ها به نام **Topic**تقسیم شده‌اند. سه API اصلی در اکوسیستم Kafka که با **انتقال داده به و از Kafka** سروکار دارند، در جدول زیر خلاصه شده‌اند.

Table 2-1. APIs for moving data to and from Kafka

API	Topic interaction	Examples
Producer API	*Writing* messages to Kafka topics.	• Filebeat • rsyslog • Custom producers
Consumer API	*Reading* messages from Kafka topics.	• Logstash • kafkacat • Custom consumers
Connect API	*Connecting* external data stores, APIs, and filesystems to Kafka topics. Involves both *reading* from topics (sink connectors) and *writing* to topics (source connectors).	• JDBC source connector • Elasticsearch sink connector • Custom connectors

جدول 1

پیش از Kafka Streams

قبل از معرفیKafka Streams ، در اکوسیستم Kafka یک **خلأ مهم** وجود داشت. این خلأ مربوط به نبود یک کتابخانه استاندارد برای پردازش جریان داده‌ها در Kafka بود که باعث می‌شد توسعه برنامه‌های **stream processing** بسیار پیچیده‌تر از حد لازم باشد.

در آن زمان، دو گزینه اصلی برای ساخت برنامه‌های پردازش جریان داده در Kafka وجود داشت:

1. **استفاده مستقیم از API های Producer و Consumer**
2. **استفاده از یک چارچوب پردازش جریان دیگر**، مانند **Apache Spark Streaming** یا **Apache Flink**

استفاده مستقیم از API های Producer و Consumer

با استفاده از این API ها می‌توان داده‌ها را مستقیماً خواند و نوشت و هر نوع پردازش دلخواه را انجام داد. اما این API ها بسیار ابتدایی هستند و فاقد بسیاری از **Primitive های پردازش جریان** مانند:

- **حالت محلی و تحمل خطا (local and fault-tolerant state)**
- مجموعه غنی‌ای از **اپراتورها برای تبدیل جریان داده‌ها**
- **نمایش‌های پیشرفته جریان‌ها**
- مدیریت پیچیده زمان (time semantics)

بنابراین، برای انجام عملیات غیرساده مانند **تجمیع داده‌ها**، **join کردن جریان‌های مختلف**، **گروه‌بندی رویدادها در پنجره‌های زمانی**، **یا اجرای کوئری‌های ad hoc روی جریان**، به سرعت با پیچیدگی زیادی مواجه می‌شوید.

استفاده از چارچوب‌های پردازش جریان کامل

گزینه دوم یعنی استفاده از Spark یا Flink ، به **پیچیدگی اضافی و سربار مدیریت یک کلاستر پردازشی** منجر می‌شود. این روش برای کارهای ساده و مستقیم با Kafka بهینه نیست، خصوصاً زمانی که می‌خواهیم با **نمایش‌های میانی داده‌ها** کار کنیم.

معرفی Kafka Streams

در سال **۲۰۱۶**، با انتشار اولین نسخه (**Kafka Streams** که گاهی به آن **Streams API** هم گفته می‌شود)، اکوسیستم Kafka تغییر بنیادینی پیدا کرد. پیش از آن، برنامه‌های پردازش جریان (stream processing) به شدت به **ویژگی‌های دستی و سفارشی** متکی بودند، اما با Kafka Streams این برنامه‌ها جای خود را به برنامه‌های پیشرفته‌تر دادند که از **الگوها و انتزاع‌های توسعه‌یافته توسط جامعه** برای پردازش و تبدیل جریان‌های داده در زمان واقعی بهره می‌بردند.

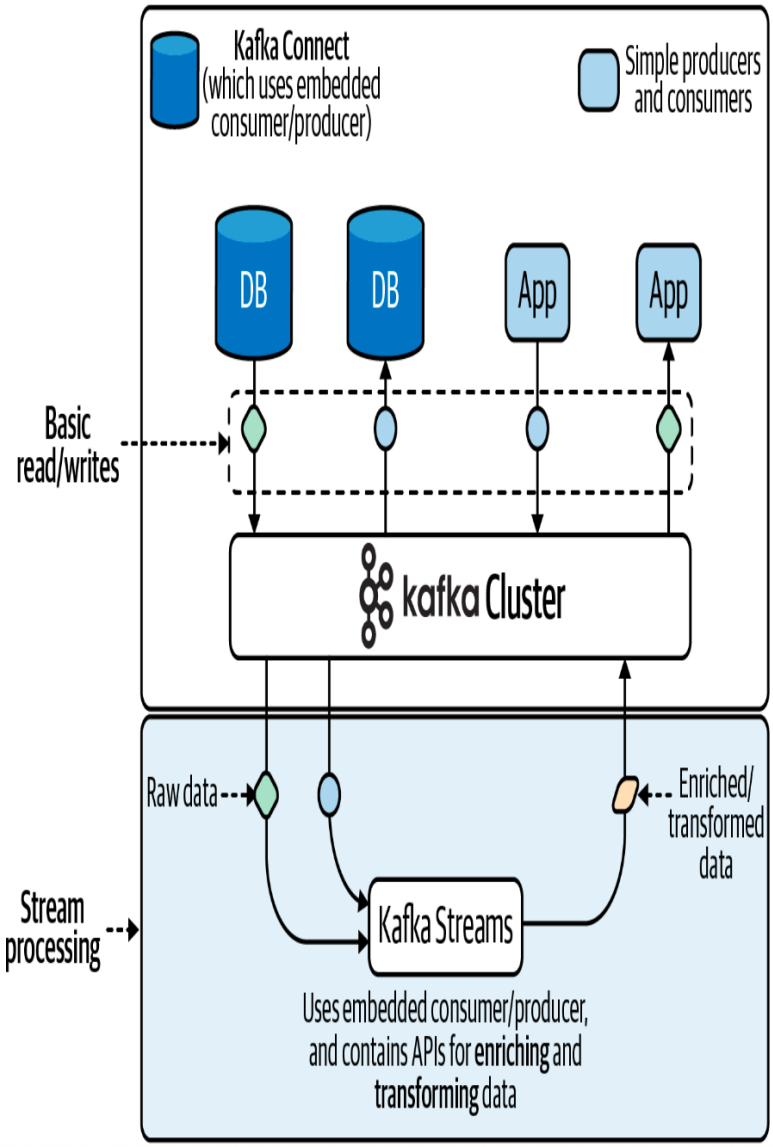
بر خلاف **APIهای Producer** ، **Consumer** و **Connect** که بیشتر برای انتقال داده‌ها به و از Kafka طراحی شده بودند، Kafka Streams **به پردازش واقعی جریان‌های داده اختصاص دارد** .این کتابخانه:

- مصرف جریان‌های داده در زمان واقعی را ساده می‌کند
- امکان **اعمال منطق تبدیل داده‌ها** با استفاده از مجموعه‌ای غنی از **اپراتورها و Primitive های پردازش جریان** را فراهم می‌کند
- به صورت اختیاری، امکان **نوشتن نسخه‌های پردازش‌شده یا غنی‌شده داده‌ها دوباره به Kafka** را می‌دهد تا سیستم‌های downstream بتوانند از آنها استفاده کنند

به عبارت دیگر، Kafka Streams این امکان را می‌دهد که نه تنها داده‌ها را منتقل کنیم، بلکه **بر روی جریان‌ها عملیات پردازشی پیچیده و غنی‌سازی داده‌ها** را به راحتی انجام دهیم.

اگر بخواهی، می‌توانم یک **نمودار ساده از نقش Kafka Streams در اکوسیستم Kafka** هم برایت بکشم تا بهتر متوجه جایگاه آن بشوی. آیا می‌خوای این را انجام دهم؟

شکل ۱-۲ API های قبلاً بحث‌شده در اکوسیستم Kafka را نشان می‌دهد، که در آن Kafka Streams در لایه پردازش جریان داده فعالیت می‌کند.



شکل 15

همان‌طور که از نمودار شکل ۱-۲ می‌بینید، Kafka Streamsدر لایه هیجان‌انگیزی از اکوسیستم Kafka فعالیت می‌کند: جایی که داده‌ها از منابع متعدد به هم می‌رسند. این لایه جایی است که می‌توان فرآیندهای پیشرفته غنی‌سازی، تبدیل و

پردازش داده‌ها را انجام داد. این همان جایی است که در دنیای پیش از Kafka Streams ، مجبور بودیم با استفاده از APIهای Consumer/Producer ، انتزاعات پردازش جریان خود را به سختی بنویسیم یا با استفاده از یک چارچوب دیگر با پیچیدگی مواجه شویم. اکنون بیایید نگاهی اولیه به ویژگی‌های Kafka Streams بیندازیم که به ما اجازه می‌دهد در این لایه به شکل جذاب و کارآمد عمل کنیم.

ویژگی‌ها به‌طور خلاصه

Kafka Streams ویژگی‌های بسیاری ارائه می‌دهد که آن را برای برنامه‌های پردازش جریان مدرن بسیار مناسب می‌کند:

- یک DSL سطح بالا که شبیه API جریان‌های جاوا است و رویکردی روان و تابعی برای پردازش جریان داده فراهم می‌کند.
- یک API پردازنده سطح پایین که کنترل دقیق برای توسعه‌دهندگان فراهم می‌کند.
- انتزاعات مناسب برای مدل‌سازی داده به‌صورت جریان یا جدول.
- قابلیت join کردن جریان‌ها و جداول، که برای تبدیل و غنی‌سازی داده‌ها مفید است.
- اپراتورها و ابزارهایی برای ساخت برنامه‌های پردازش جریان بدون حالت و با حالت.
- پشتیبانی از عملیات مبتنی بر زمان، شامل windowing و توابع دوره‌ای.
- نصب آسان؛ فقط یک کتابخانه است و می‌توان Kafka Streams را به هر برنامه جاوا اضافه کرد.
- مقیاس‌پذیری، قابلیت اطمینان، و نگهداری‌پذیری.

همچنین، APIسطح بالا و API پردازنده سطح پایین هم یادگیری آسان دارند و هم بسیار قدرتمند هستند. انجام وظایف پیشرفته پردازش جریان (مثل join کردن جریان‌های زنده و در حال حرکت) با کد بسیار کمی امکان‌پذیر است.

ویژگی‌های عملیاتی

سیستم‌ها زمانی مقیاس‌پذیر محسوب می‌شوند که با افزایش بار کاری، عملکرد خود را حفظ کنند. در Kafka ، مقیاس‌گذاری موضوعات با افزودن پارتیشن‌ها و در صورت نیاز، افزودن broker های بیشتر انجام می‌شود. در Kafka Streams ، واحد کار یک پارتیشن از یک topic است و Kafka به‌طور خودکار کار را به گروه‌های مصرف‌کننده (consumer groups) توزیع می‌کند.

دو نکته مهم:

1. از آنجا که واحد کار یک پارتیشن است و می‌توان با افزودن پارتیشن‌های بیشتر، موضوعات را گسترش داد، میزان کاری که یک برنامه Kafka Streams می‌تواند انجام دهد با افزایش تعداد پارتیشن‌ها قابل افزایش است.
2. با استفاده از consumer groups ، کل حجم کاری که توسط برنامه Kafka Streams انجام می‌شود می‌تواند بین نمونه‌های متعدد و همکار برنامه توزیع شود.

Kafka Streams همچنین انعطاف‌پذیر است و امکان مقیاس‌بندی دستی تعداد نمونه‌های برنامه را فراهم می‌کند، تا با افزایش یا کاهش بار کاری سازگار باشد.

مقیاس‌پذیری

سیستم‌ها زمانی مقیاس‌پذیر محسوب می‌شوند که بتوانند با افزایش بار کاری، عملکرد خود را حفظ کنند. در فصل ۱ آموختیم که مقیاس‌گذاری موضوعات Kafka با افزودن پارتیشن‌های بیشتر و در صورت نیاز، افزودنbroker های بیشتر انجام می‌شودbroker) های اضافی تنها زمانی لازم هستند که موضوع از ظرفیت موجود خوشه Kafka شما فراتر رود.(

به‌طور مشابه، درKafka Streams ، واحد کار یک پارتیشن از یک topic است و Kafka به‌طور خودکار کار را به گروه‌های مصرف‌کننده همکاری‌کننده (consumer groups) توزیع می‌کند.

این دو نکته مهم را به همراه دارد:

1. از آنجا که واحد کار یک پارتیشن است و موضوعات با افزودن پارتیشن‌های بیشتر قابل گسترش هستند، میزان کاری که یک برنامه Kafka Streams می‌تواند انجام دهد با افزایش تعداد پارتیشن‌ها روی موضوعات منبع، قابل افزایش است.

2. با بهره‌گیری ازconsumer groups ، کل حجم کاری که توسط یک برنامه Kafka Streams انجام می‌شود می‌تواند بین چند نمونه همکاری‌کننده از برنامه شما توزیع شود.

یک نکته کوتاه درباره مورد دوم: هنگام استقرار یک برنامهKafka Streams ، معمولاً چندین نمونه از برنامه را مستقر می‌کنید که هر کدام بخشی از کار را مدیریت می‌کنند (مثلاً اگر topic منبع شما ۳۲ پارتیشن دارد، آنگاه ۳۲ واحد کار وجود دارد که می‌تواند بین همه مصرف‌کنندگان همکاری‌کننده توزیع شود). برای مثال، می‌توانید چهار نمونه برنامه مستقر کنید که هر کدام هشت پارتیشن را مدیریت کنند (۴ * ۸ = ۳۲) یا به‌طور مشابه، شانزده نمونه از برنامه خود مستقر کنید که هر کدام دو پارتیشن را مدیریت کنند (۱۶ * ۲ = ۳۲.(

با این حال، صرف نظر از تعداد نمونه‌های برنامه‌ای که مستقر می‌کنید، توانایی Kafka Streams در مقابله با بار افزایش‌یافته با افزودن پارتیشن‌های بیشتر (واحدهای کار) و نمونه‌های برنامه (کارگران) آن را مقیاس‌پذیر می‌کند.

به‌طور مشابه، Kafka Streamsهمچنین **انعطاف‌پذیر** است و امکان مقیاس‌بندی دستی تعداد نمونه‌های برنامه را به صورت روان فراهم می‌کند، با این محدودیت که مسیر scale-out تا تعدادtask هایی که برای توپولوژی شما ایجاد شده است محدود است. (جزئیات بیشتر دربارهtask ها در بخش «Tasks and Stream Threads» بررسی خواهد شد(.

قابلیت اطمینان

قابلیت اطمینان یک ویژگی مهم سیستم‌های داده‌ای است، نه تنها از دیدگاه مهندسی (ما نمی‌خواهیم ساعت ۳ صبح به دلیل خطا در سیستم بیدار شویم)، بلکه از دیدگاه مشتریان ما نیز (نمی‌خواهیم سیستم به‌طور قابل توجهی آفلاین شود و قطعاً نمی‌توانیم از دست رفتن یا فساد داده را تحمل کنیم Kafka Streams .(دارای چند ویژگی تحمل خطا است، اما واضح‌ترین آن چیزی است که قبلاً در بخش «Consumer Groups» به آن اشاره شد **failover :خودکار و تعادل مجدد پارتیشن‌ها از طریق گروه‌های مصرف‌کننده.**

اگر چندین نمونه از برنامه Kafka Streams خود را مستقر کنید و یکی به دلیل خطا در سیستم (مثلاً خرابی سخت‌افزار) آفلاین شود، Kafkaبه‌طور خودکار بار کاری را به سایر نمونه‌های سالم توزیع می‌کند. هنگامی که خطا برطرف شد (یا در معماری‌های مدرن که از سیستم‌های ارکستراسیون مانند Kubernetes استفاده می‌کنند، وقتی برنامه به یک گره سالم منتقل می‌شود(، Kafkaمجدداً کار را متعادل می‌کند. این قابلیت مدیریت ملایم خطاها باعث می‌شود Kafka Streams قابل اعتماد باشد.

قابلیت نگهداری

مشخص است که بیشتر هزینه‌های نرم‌افزار نه در توسعه اولیه، بلکه در نگهداری مداوم آن است—رفع باگ‌ها، حفظ عملیات سیستم، بررسی خطاها...

—مارتین کلپمن

از آنجا که Kafka Streams یک کتابخانه جاوا است، رفع مشکلات و باگ‌ها نسبتاً ساده است زیرا با برنامه‌های مستقل کار می‌کنیم و الگوهای عیب‌یابی و مانیتورینگ برنامه‌های جاوا به خوبی تثبیت شده و ممکن است در سازمان شما از قبل استفاده شوند (جمع‌آوری و تحلیل لاگ‌ها، ثبت متریک‌های برنامه وJVM ، پروفایلینگ و ردیابی و غیره.(

علاوه بر این، از آنجا که API Kafka Streams مختصر و شهودی است، نگهداری کد کمتر وقت‌گیر است نسبت به کتابخانه‌های پیچیده‌تر و برای مبتدیان و کارشناسان به یک اندازه آسان برای درک است. اگر یک برنامه Kafka Streamsبسازید و ماه‌ها به آن دست نزنید، احتمالاً با فراموشی معمول پروژه مواجه نخواهید شد و نیازی به زمان زیادی برای درک کد قبلی نخواهید داشت. به همین دلایل، نگهدارندگان جدید پروژه می‌توانند به سرعت با برنامه Kafka Streamsآشنا شوند که قابلیت نگهداری را حتی بیشتر بهبود می‌بخشد.

مقایسه با سایر سیستم‌ها

تا اینجا، باید شروع به احساس راحتی برای شروع یک رابطه بلندمدت با Kafka Streams کرده باشید. اما قبل از اینکه همه چیز جدی شود، طبیعی نیست که ببینیم آیا گزینه‌های دیگری هم وجود دارند؟

در واقع، گاهی اوقات ارزیابی کیفیت یک فناوری بدون دانستن اینکه در مقایسه با رقبای خود چگونه عمل می‌کند دشوار است. بنابراین بیایید ببینیم Kafka Streams در مقایسه با برخی فناوری‌های محبوب دیگر در حوزه پردازش جریان چگونه است. ابتدا مدل استقرار Kafka Streams را با سایر سیستم‌های محبوب مقایسه می‌کنیم.

مدل استقرار

Kafka Streamsرویکرد متفاوتی نسبت به پردازش جریان در مقایسه با فناوری‌هایی مانند Apache Flink و Apache Spark Streamingدارد. این سیستم‌ها نیاز دارند که یک کلاستر پردازشی اختصاصی برای ارسال و اجرای برنامه پردازش جریان خود راه‌اندازی کنید. این کار می‌تواند پیچیدگی و سربار زیادی ایجاد کند. حتی مهندسان با تجربه در شرکت‌های شناخته‌شده اذعان کرده‌اند که سربار کلاستر پردازشی غیرقابل چشم‌پوشی است.

از سوی دیگر، Kafka Streamsبه‌عنوان یک کتابخانه جاوا پیاده‌سازی شده است، بنابراین شروع کار بسیار آسان‌تر است زیرا نیازی به نگرانی درباره مدیر کلاستر ندارید؛ تنها کافی است وابستگی Kafka Streams را به برنامه جاوای خود اضافه کنید. توانایی ساخت برنامه پردازش جریان به‌عنوان یک برنامه مستقل همچنین به شما آزادی زیادی در نحوه مانیتورینگ، بسته‌بندی و استقرار کد می‌دهد. برای مثال، درMailchimp ، برنامه‌های Kafka Streams ما با همان الگوها و ابزارهایی که برای سایر برنامه‌های داخلی جاوا استفاده می‌کنیم مستقر می‌شوند. این توانایی برای ادغام فوری با سیستم‌های شرکت، مزیت بزرگی برای Kafka Streams است.

مدل پردازش

یکی دیگر از تفاوت‌های کلیدی بین Kafka Streams و سیستم‌هایی مانند Apache Spark Streaming یا Trident این است که Kafka Streams **پردازش رویداد به‌ازای هر رویداد** را پیاده‌سازی می‌کند، بنابراین رویدادها به‌صورت فوری و یکی‌یکی در زمان ورود پردازش می‌شوند. این روش پردازش جریان واقعی محسوب می‌شود و تاخیر کمتری نسبت به روش جایگزین دارد، که به آن **میکروبچینگ** می‌گویند. میکروبچینگ شامل گروه‌بندی رکوردها در دسته‌های

کوچک (یا batch) اســت که در حافظه بافر شــده و بعداً در فواصــل زمانی مشــخص (مثلاً هر ۵۰۰ میلی‌ثانیه) منتشــر می‌شوند.

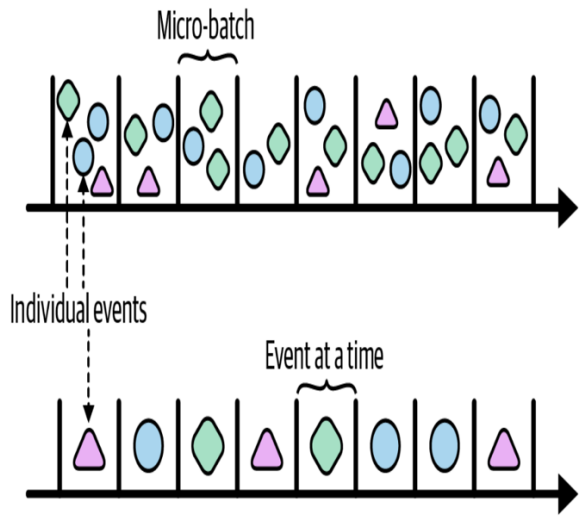


Figure 2-2. Micro-batching involves grouping records into small batches and emitting them to downstream processors at a fixed interval; event-at-a-time processing allows each event to be processed at soon as it comes in, instead of waiting for a batch to materialize

توجه!

چارچوب‌هایی که از میکروبچینگ اســتفاده می‌کنند اغلب برای دســتیابی به توان عملیاتی بالاتر به هزینه تاخیر بیشــتر بهینه‌سازی شده‌اند. درKafka Streams ، می‌توانید تاخیر بسیار پایین را همزمان با حفظ توان عملیاتی بالا با تقسیم داده‌ها در چندین پارتیشن به‌دست آورید.

معماری کاپا(Kappa Architecture)

یکی دیگر از ملاحظات مهم هنگام مقایسه Kafka Streams با سایر راه‌حل‌ها این است که آیا مورد استفاده شما نیاز به پشــتیبانی همزمان از پردازش دســته‌ای (Batch) و جریان (Stream) دارد یا خیر. در زمان نگارش این متن، Kafka Streamsصــرفاً بر روی پردازش جریان تمرکز دارد (این را معماری کاپا می‌نامند)، در حالی که چارچوب‌هایی مانند Apache Flinkو Apache Spark هم پردازش دسته‌ای و هم پردازش جریان را پشتیبانی می‌کنند (این را معماری لمبدا می‌نامند.(

با این حال، معماری‌هایی که هر دو حالت دسته‌ای و جریان را پشتیبانی می‌کنند بدون معایب نیستند Jay Kreps .تقریباً دو سال قبل از معرفی Kafka Streams در اکوسیستمKafka ، برخی معایب سیستم‌های هیبریدی را مطرح کرد:

- بار عملیاتی اجرای دو سیستم و عیب‌یابی آن بسیار بالا خواهد بود.
- هر انتزاع جدید تنها می‌تواند ویژگی‌هایی را ارائه دهد که در تقاطع هر دو سیستم پشتیبانی می‌شوند.

این چالش‌ها مانع محبوبیت پروژه‌هایی مانندApache Beam ، که یک مدل برنامه‌نویسی یکپارچه برای پردازش دسته‌ای و جریان ارائه می‌دهد، نشــده اســت. اما Apache Beam به همان شــیوه‌ای که Apache Flink مقایســه می‌شــود، قابل مقایسه با Kafka Streams نیست. در عوض، Apache Beamیک لایه API است که برای انجام بیشتر کارها به یک

موتور اجرایی وابسته است. به‌عنوان مثال، هم Apache Flink و هم Apache Spark می‌توانند به‌عنوان موتور اجرایی (که اغلب به آن runner گفته می‌شود) در Apache Beam استفاده شوند. بنابراین هنگام مقایسه Kafka Streams با Apache Beam، باید موتور اجرایی مورد استفاده را علاوه بر خود API Beam در نظر گرفت.

علاوه بر این، پایپلاین‌های مبتنی بر Apache Beam برخی ویژگی‌های مهم که در Kafka Streams ارائه می‌شوند را ندارند. Robert Yokota، که یک Kafka Streams Beam Runner تجربی ایجاد کرده و چندین پروژه نوآورانه در اکوسیستم Kafka را نگهداری می‌کند، تفاوت این دو چارچوب را این‌گونه بیان می‌کند:

- Kafka Streams یک پلتفرم پردازش جریان-رابطه‌ای است.
- Apache Beam یک پلتفرم پردازش فقط جریان است.

یک پلتفرم پردازش جریان-رابطه‌ای دارای قابلیت‌هایی است که معمولاً در یک پلتفرم پردازش فقط جریان وجود ندارد:

- روابط (یا جداول) شهروندان درجه‌یک هستند، یعنی هر کدام هویت مستقلی دارند.
- روابط می‌توانند به روابط دیگر تبدیل شوند.
- روابط می‌توانند به‌صورت ad-hoc پرس‌وجو شوند.

ما هر یک از این ویژگی‌ها را در چند فصل بعدی نشان خواهیم داد، اما فعلاً کافی است بدانید که بسیاری از قدرتمندترین ویژگی‌های Kafka Streams از جمله قابلیت پرس‌وجوی وضعیت یک جریان) در Apache Beam یا سایر چارچوب‌های عمومی‌تر موجود نیستند. علاوه بر این، معماری کاپا رویکرد ساده‌تر و تخصصی‌تری برای کار با جریان‌های داده ارائه می‌دهد، که می‌تواند تجربه توسعه را بهبود بخشد و عملیات و نگهداری نرم‌افزار را ساده‌تر کند. بنابراین اگر مورد استفاده شما نیاز به پردازش دسته‌ای ندارد، سیستم‌های هیبریدی پیچیدگی غیرضروری ایجاد می‌کنند.

حالا که به رقبا نگاه کردیم، بیایید برخی از موارد استفاده Kafka Streams را بررسی کنیم.

موارد استفاده

Kafka Streams برای پردازش سریع و کارآمد داده‌های نامحدود بهینه‌سازی شده است و بنابراین یک راه‌حل عالی برای مسائل حوزه‌هایی با تاخیر پایین و زمان‌حساس محسوب می‌شود. چند نمونه از موارد استفاده عبارت‌اند از:

- پردازش داده‌های مالی(Flipkart) ، نظارت بر خرید، تشخیص تقلب
- معامله الگوریتمی
- نظارت بر بازار سهام/صرافی ارزهای دیجیتال
- ردیابی و تجدید موجودی لحظه‌ای(Walmart)
- رزرو رویداد و انتخاب صندلی(Ticketmaster)
- پیگیری و نظارت بر تحویل ایمیل(Mailchimp)
- پردازش تله‌متری بازی‌های ویدیویی(Activision) ، ناشر(Call of Duty
- ایندکس‌گذاری جستجو(Yelp)

- ردیابی/محاسبات جغرافیایی (مثلاً مقایسه فاصله، پیش‌بینی زمان رسیدن(
- پردازش حسگرهای خانه هوشمند) IoT/گاهی AIOT یا هوش مصنوعی اشیا(
- تغییر داده‌ها و ضبط تغییرات(Redhat)
- پخش ورزشی/ویجت‌های لحظه‌ای(Gracenote)
- پلتفرم‌های تبلیغات لحظه‌ای(Pinterest)
- مراقبت‌های بهداشتی پیش‌بینی‌کننده، نظارت بر علائم حیاتی(Children's Healthcare of Atlanta)
- زیرساخت چت(Slack) ، ربات‌های چت، دستیاران مجازی
- پایپلاین‌ها و پلتفرم‌های یادگیری ماشین(Twitter) ، (Kafka Graphs

ویژگی مشترک تمام این نمونه‌ها این است که آن‌ها نیازمند تصمیم‌گیری یا پردازش داده در زمان واقعی هستند. گستره این موارد استفاده، از تحلیل خروجی حسگر یک دستگاه خانه هوشمند در سطح سرگرمی تا نظارت و واکنش به تغییرات وضعیت یک بیمار در مراکز درمانی، بسیار جالب است.

Kafka Streamsهمچنین گزینه‌ای عالی برای ساخت میکروسرویس‌ها روی جریان‌های داده زمان واقعی است. این کتابخانه نه تنها عملیات معمول پردازش جریان (فیلتر، اتصال، پنجره‌بندی، و تبدیل داده‌ها) را ساده می‌کند، بلکه همان‌طور که در بخش «Interactive Queries» خواهید دید، قابلیت نمایش وضعیت جریان با استفاده از ویژگی به نام پرس‌وجوهای تعاملی را نیز دارد. وضعیت یک جریان می‌تواند جمع‌آوری نوعی داده باشد (مثلاً تعداد کل بازدیدها برای هر ویدیو در یک پلتفرم استریمینگ) یا حتی آخرین نمایه یک موجودیت در حال تغییر سریع در جریان رویداد شما (مثلاً آخرین قیمت سهام یک نماد خاص.(

پردازشگرها و توپولوژی‌ها

Kafka Streamsاز پارادایم برنامه‌نویسی به نام برنامه‌نویسی جریان داده (Dataflow Programming - DFP) استفاده می‌کند، که روشی داده‌محور برای نمایش برنامه‌ها به‌صورت مجموعه‌ای از ورودی‌ها، خروجی‌ها و مراحل پردازش است. این رویکرد یک روش طبیعی و شهودی برای ایجاد برنامه‌های پردازش جریان فراهم می‌کند و یکی از دلایل آسان بودن یادگیری Kafka Streams برای مبتدیان است.

به جای ساختن برنامه به‌صورت یک توالی مراحل، منطق پردازش جریان در یک برنامه Kafka Streams به‌صورت یک گراف جهت‌دار غیرمدور (DAG) ساختاربندی می‌شود. در این گراف، گره‌ها (مستطیل‌ها در نمودار) نمایانگر یک مرحله پردازش یا پردازنده هستند و یال‌ها (خط‌های متصل‌کننده گره‌ها) جریان ورودی و خروجی را نشان می‌دهند، یعنی داده‌ها از یک پردازنده به پردازنده دیگر منتقل می‌شوند.

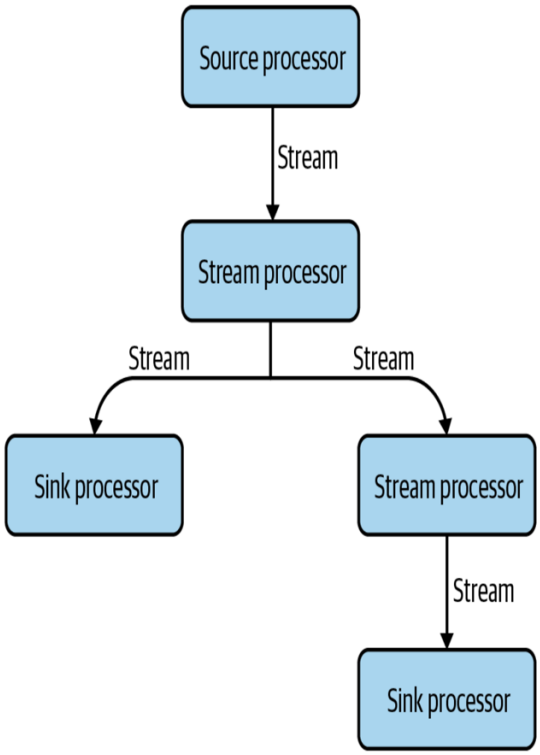


Figure 2-3. Kafka Streams borrows some of its design from dataflow programming, and structures stream processing programs as a graph of processors through

شکل 16

در Kafka Streams سه نوع پردازنده پایه وجود دارد:

پردازنده‌های منبع(Source processors)

منابع جایی هستند که اطلاعات وارد برنامه Kafka Streams می‌شوند. داده‌ها از یک موضوع Kafka خوانده شده و به یک یا چند پردازنده جریان ارسال می‌شوند.

پردازنده‌های جریان(Stream processors)

این پردازنده‌ها مسئول اعمال منطق پردازش یا تبدیل داده روی جریان ورودی هستند. در DSL سطح بالا، این پردازنده‌ها با استفاده از مجموعه‌ای از عملگرهای از پیش تعریف‌شده که توسط کتابخانه Kafka Streams ارائه می‌شوند، تعریف می‌شوند. برخی از مثال‌های عملگرها عبارت‌اند از filter :، map، flatMap و join.

پردازنده‌های مقصد(Sink processors)

مقصدها جایی هستند که رکوردهای پردازش‌شده، غنی‌شده یا فیلترشده دوباره به Kafka نوشته می‌شوند، تا توسط برنامه دیگری پردازش شوند یا به یک مخزن داده پایین‌دستی از طریق چیزی مانند Kafka Connect ارسال شوند. مشابه پردازنده‌های منبع، پردازنده‌های مقصد نیز به یک موضوع Kafka متصل هستند.

مجموعه‌ای از پردازنده‌ها یک توپولوژی پردازنده را تشکیل می‌دهد، که معمولاً در این کتاب و جامعه Kafka Streams تنها با نام توپولوژی شناخته می‌شود. در هر آموزش این بخش، ابتدا توپولوژی با ایجاد یک DAG که پردازنده‌های منبع، جریان و مقصد را به هم متصل می‌کند، طراحی می‌شود و سپس توپولوژی با نوشتن کد Java پیاده‌سازی می‌شود. این روش به شما کمک می‌کند تا مانند یک توسعه‌دهنده Kafka Streams فکر کنید.

در Kafka Streams سه نوع پردازنده پایه وجود دارد:

پردازنده‌های منبع(Source processors)

منابع جایی هستند که اطلاعات وارد برنامه Kafka Streams می‌شوند. داده‌ها از یک موضوع Kafka خوانده شده و به یک یا چند پردازنده جریان ارسال می‌شوند.

پردازنده‌های جریان(Stream processors)

این پردازنده‌ها مسئول اعمال منطق پردازش یا تبدیل داده روی جریان ورودی هستند. در DSL سطح بالا، این پردازنده‌ها با استفاده از مجموعه‌ای از عملگرهای از پیش تعریف‌شده که توسط کتابخانه Kafka Streams ارائه می‌شوند، تعریف می‌شوند. برخی از مثال‌های عملگرها عبارت‌اند از filter :، map، flatMap و join.

پردازنده‌های مقصد(Sink processors)

مقصدها جایی هستند که رکوردهای پردازش‌شده، غنی‌شده یا فیلترشده دوباره به Kafka نوشته می‌شوند، تا توسط برنامه دیگری پردازش شوند یا به یک مخزن داده پایین‌دستی از طریق چیزی مانند Kafka Connect ارسال شوند. مشابه پردازنده‌های منبع، پردازنده‌های مقصد نیز به یک موضوع Kafka متصل هستند.

مجموعه‌ای از پردازنده‌ها یک توپولوژی پردازنده را تشکیل می‌دهد، که معمولاً در این کتاب و جامعه Kafka Streams تنها با نام توپولوژی شناخته می‌شود. در هر آموزش این بخش، ابتدا توپولوژی با ایجاد یک DAG که پردازنده‌های منبع، جریان و مقصد را به هم متصل می‌کند، طراحی می‌شود و سپس توپولوژی با نوشتن کد Java پیاده‌سازی می‌شود. این روش به شما کمک می‌کند تا مانند یک توسعه‌دهنده Kafka Streams فکر کنید.

سناریو:

فرض کنید در حال ساخت یک چت‌بات با Kafka Streams هستیم و یک موضوع به نام **slack-mentions** داریم که شامل تمام پیام‌های Slack است که به ربات ما، **@StreamsBot** اشاره دارند. ربات ما طوری طراحی می‌شود که انتظار دارد هر اشاره با یک دستور همراه باشد، مانند:

```
@StreamsBot restart myservice
```

ما می‌خواهیم یک توپولوژی پردازنده پایه پیاده‌سازی کنیم که پیش‌پردازش و اعتبارسنجی پیام‌های Slack را انجام دهد. ابتدا باید هر پیام در موضوع منبع را مصرف کنیم، بررسی کنیم که آیا دستور معتبر است یا خیر، و در صورت معتبر بودن، پیام Slack را به موضوع **valid-mentions** بنویسیم. اگر دستور نامعتبر باشد (مثلاً کاربری دستور را اشتباه نوشته باشد، مانند @StreamsBot restart serverrr)، پیام به موضوع **invalid-mentions** نوشته می‌شود.

در این حالت، این نیازمندی‌ها به توپولوژی‌ای مشابه شکل **Figure 2-4** ترجمه می‌شوند.

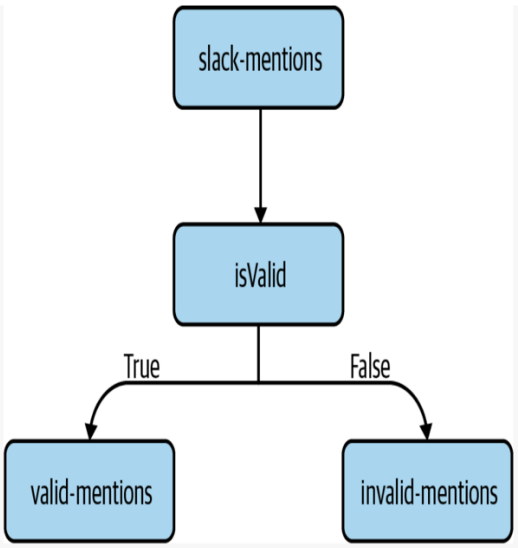


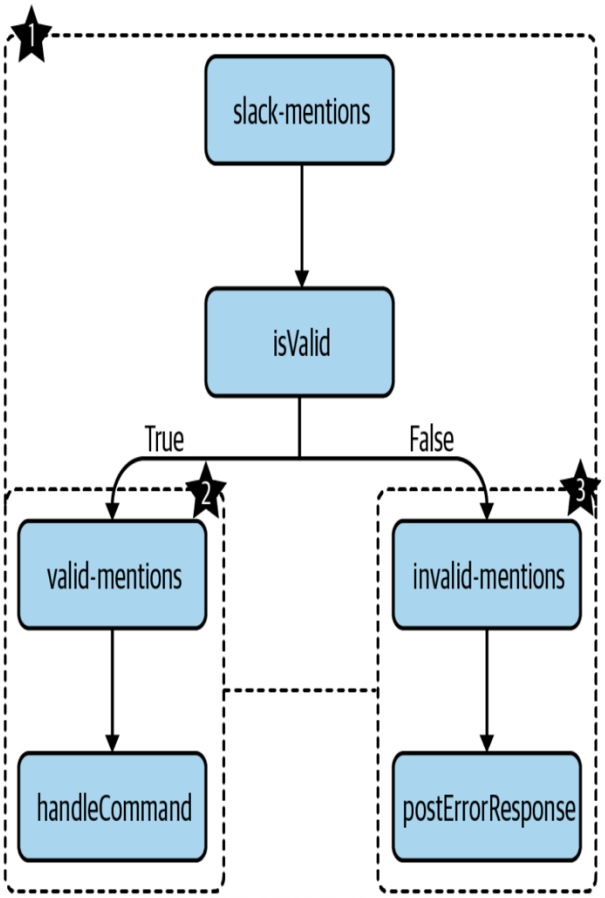
Figure 2-4. An example processor topology that contains a single source processor for reading Slack messages from Kafka (`slack-mentions`), a single stream processor that checks the validity of each message (`isValid`), and two sink processors that route the message to one of two output topics based on the previous check (`valid-mentions`, `invalid-mentions`)

زیر-توپولوژی‌ها:

Kafka Streams همچنین مفهوم **زیر-توپولوژی‌ها** را دارد. در مثال قبلی، ما یک توپولوژی پردازنده طراحی کردیم که رویدادها را از یک موضوع منبع (**slack-mentions**) مصرف می‌کند و پیش‌پردازشی روی جریان پیام‌های خام چت انجام می‌دهد. با این حال، اگر برنامه ما نیاز داشته باشد که از چندین موضوع منبع مصرف کند، Kafka Streamsدر بیشتر موارد توپولوژی را به بخش‌های کوچکتر یا **زیر-توپولوژی‌ها** تقسیم می‌کند تا کارها را به صورت موازی بیشتر انجام دهد. این تقسیم‌بندی امکان‌پذیر است زیرا عملیات روی یک جریان ورودی می‌تواند مستقل از عملیات روی جریان دیگر اجرا شود.

به عنوان مثال، اگر چت‌بات خود را با افزودن دو پردازنده جریان جدید توسعه دهیم: یکی که از موضوع **valid-mentions**مصرف می‌کند و دستور صادر شده به StreamsBot را اجرا می‌کند (مثلاً restart server) ، و دیگری که از موضوع **invalid-mentions**مصرف می‌کند و پاسخ خطا را به Slack ارسال می‌کند.

همان‌طور که در شکل **Figure 2-5**دیده می‌شود، توپولوژی ما اکنون سه موضوع Kafka می‌خواند: **slack-mentions**، **valid-mentions**و **invalid-mentions**. هر بار که از یک موضوع منبع جدید می‌خوانیم، Kafka Streamsتوپولوژی را به بخش‌های کوچکتری تقسیم می‌کند که می‌تواند به‌صورت مستقل اجرا شود. در این مثال، سه زیر-توپولوژی برای برنامه چت‌بات ایجاد می‌شود، که هر کدام با یک ستاره در شکل نشان داده شده‌اند.



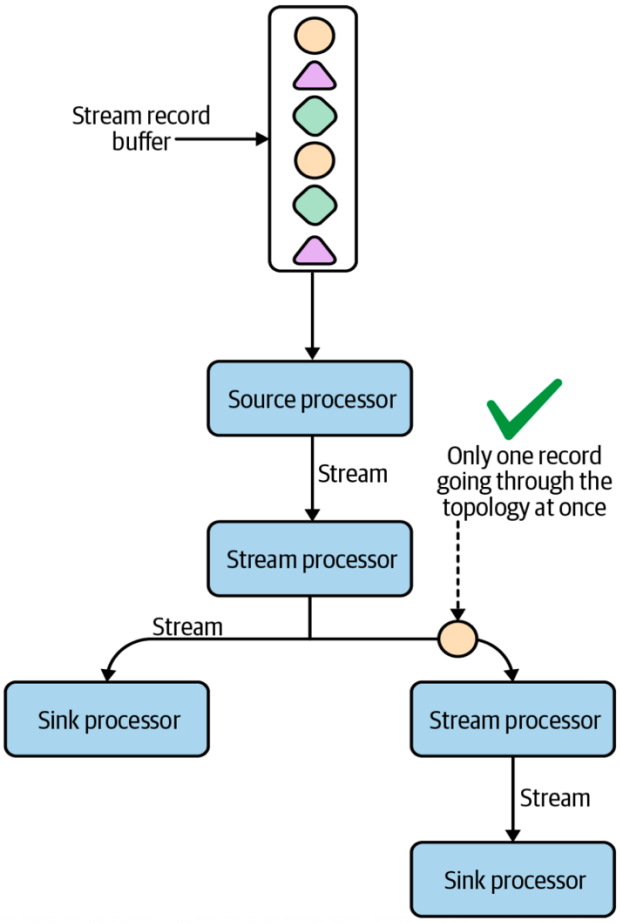
شکل 17

توجه داشــته باشــید که هر دو موضــوع **valid-mentions**و **invalid-mentions**به‌عنوان یک پردازنده **sink**در زیر-توپولوژی اول عمل می‌کنند، اما به‌عنوان پردازنده **source**در زیر-توپولوژی‌های دوم و ســـوم عمل می‌کنند. وقتی این اتفاق می‌افتد، هیچ تبادل داده مســـتقیمی بین زیر-توپولوژی‌ها صـــورت نمی‌گیرد. رکوردها ابتدا در پردازنده sink به Kafkaتولید می‌شوند و سپس توسط پردازنده‌های source دوباره از Kafka خوانده می‌شوند.

حال که می‌دانیم چگونه یک برنامه پردازش جریان را به شـــکل توپولوژی پردازنده نمایش دهیم، بیایید ببینیم داده‌ها چگونه در پردازنده‌های متصل به هم در یک برنامه Kafka Streams جریان می‌یابند.

پردازش عمق-اول(Depth-First Processing)

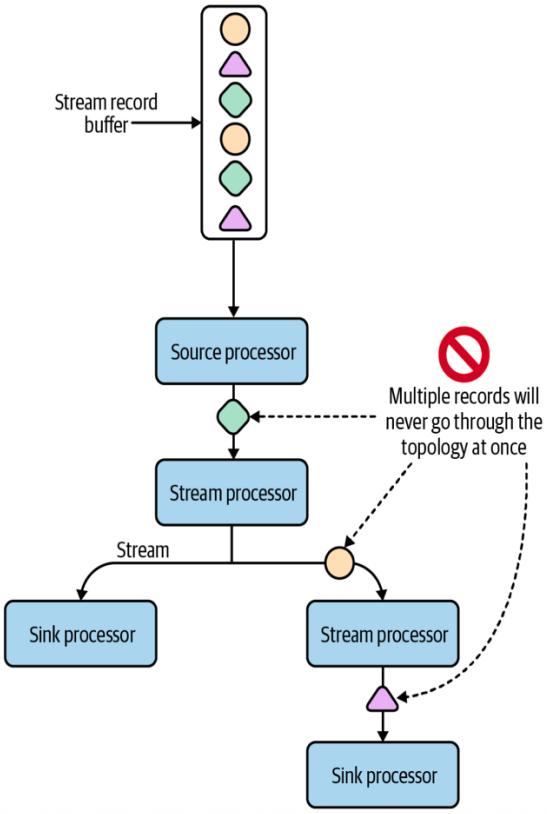
Kafka Streamsهنگام پردازش داده‌ها از استراتژی **عمق-اول** استفاده می‌کند. وقتی یک رکورد جدید دریافت می‌شود، قبل از پردازش رکورد بعدی، از طریق هر پردازنده جریان در توپولوژی هدایت می‌شـــود. جریان داده‌ها در Kafka Streamsشبیه آن چیزی است که در شکل **Figure 2-6**نشان داده شده است.



شکل 18

این استراتژی **عمق-اول** باعث می‌شود جریان داده‌ها راحت‌تر قابل درک باشد، اما همچنین بدین معناست که عملیات کند پردازش جریان می‌تواند سایر رکوردها را در همان رشته (thread) از پردازش باز دارد.

شکل **Figure 2-7** چیزی را نشان می‌دهد که هرگز در Kafka Streams رخ نمی‌دهد: پردازش همزمان چند رکورد در یک توپولوژی.



شکل 19

مزایای برنامه‌نویسی جریان داده:

استفاده از Kafka Streams و مدل برنامه‌نویسی جریان داده برای ساخت برنامه‌های پردازش جریان مزایای متعددی دارد. اولاً، نمایش برنامه به‌صورت یک **گراف جهت‌دار** باعث می‌شود فهم آن آسان باشد. نیازی نیست برای فهم جریان داده‌ها در برنامه، مجموعه‌ای از شرط‌ها و منطق کنترلی را دنبال کنید. کافی است پردازنده‌های منبع و مقصد را پیدا کنید تا بدانید داده‌ها از کجا وارد و خارج می‌شوند، و پردازنده‌های میانی را بررسی کنید تا بفهمید داده‌ها چگونه پردازش، تبدیل و غنی‌سازی می‌شوند.

علاوه بر این، بیان برنامه پردازش جریان به‌صورت گراف جهت‌دار اجازه می‌دهد روش استانداردی برای چارچوب‌بندی مسائل پردازش داده‌های زمان واقعی و ساخت راه‌حل‌های جریان داده داشته باشیم. برنامه Kafka Streams که من می‌نویسم، برای هر کسی که قبلاً با Kafka Streams کار کرده است آشنا خواهد بود — نه تنها به‌خاطر انتزاعات قابل استفاده مجدد در کتابخانه، بلکه به‌خاطر رویکرد مشترک حل مسئله: تعریف جریان داده با استفاده از **اپراتورها (گره‌ها)** و **جریان‌ها (یال‌ها)**. این موضوع باعث می‌شود برنامه‌های Kafka Streams قابل فهم و نگهداری باشند.

گراف‌های جهت‌دار همچنین روشی شهودی برای نمایش جریان داده‌ها برای ذینفعان غیر فنی هستند. اغلب یک فاصله بین تیم‌های مهندسی و تیم‌های غیرمهندسی در درک نحوه عملکرد برنامه وجود دارد. گاهی این باعث می‌شود تیم‌های غیر فنی نرم‌افزار را یک جعبه سیاه بدانند. این می‌تواند اثرات جانبی خطرناکی داشته باشد، به‌خصوص در عصر قوانین حفظ حریم خصوصی داده و رعایت GDPR که نیاز به هماهنگی نزدیک بین مهندسین، تیم‌های حقوقی و سایر ذینفعان دارد. بنابراین،

توانایی توضیح ساده نحوه پردازش داده‌ها در برنامه، اجازه می‌دهد افراد متمرکز بر جنبه‌های دیگر مشکل کسب‌وکار، طراحی برنامه را درک یا حتی در آن مشارکت کنند.

در نهایت، **توپولوژی پردازنده** که شامل پردازنده‌های منبع، مقصد و جریان است، به‌عنوان یک قالب عمل می‌کند که می‌توان به‌راحتی آن را در چندین رشته و نمونه برنامه تکرار و موازی‌سازی کرد. بنابراین، تعریف جریان داده به این شکل باعث می‌شود بتوانیم از مزایای عملکرد و مقیاس‌پذیری بهره ببریم، زیرا می‌توان برنامه پردازش جریان را به‌سادگی هنگام افزایش حجم داده‌ها تکثیر کرد.

برای درک نحوه تکثیر توپولوژی‌ها، ابتدا باید رابطه بین **تسک‌ها، رشته‌های جریان و پارتیشن‌ها** را بفهمیم.

تسک‌ها و رشته‌های جریان:

هنگامی که در Kafka Streams یک توپولوژی تعریف می‌کنیم، در واقع برنامه را اجرا نمی‌کنیم، بلکه یک **قالب** برای جریان داده‌ها در برنامه ایجاد می‌کنیم. این قالب (توپولوژی ما) می‌تواند چندین بار در یک نمونه برنامه ایجاد شود و بین **تسک‌ها و رشته‌های جریان**)که از این به بعد به‌سادگی «رشته» نامیده می‌شوند) به‌صورت موازی اجرا شود. رابطه نزدیکی بین تعداد تسک‌ها/رشته‌ها و مقدار کاری که برنامه پردازش جریان شما می‌تواند انجام دهد وجود دارد، بنابراین درک این بخش برای رسیدن به عملکرد خوب در Kafka Streams اهمیت زیادی دارد.

تسک‌ها:

یک تسک کوچک‌ترین واحد کاری است که می‌تواند به‌صورت موازی در یک برنامه Kafka Streams انجام شود.

به‌صورت ساده، بیشینه موازی‌سازی که برنامه شما می‌تواند داشته باشد توسط بیشینه تعداد **تسک‌های جریان** محدود می‌شود، که خود این تعداد توسط بیشینه تعداد پارتیشن‌های موضوع ورودی‌ای که برنامه می‌خواند تعیین می‌شود.

فرمول محاسبه تعداد تسک‌ها برای یک زیر-توپولوژی Kafka Streams به شکل زیر است:

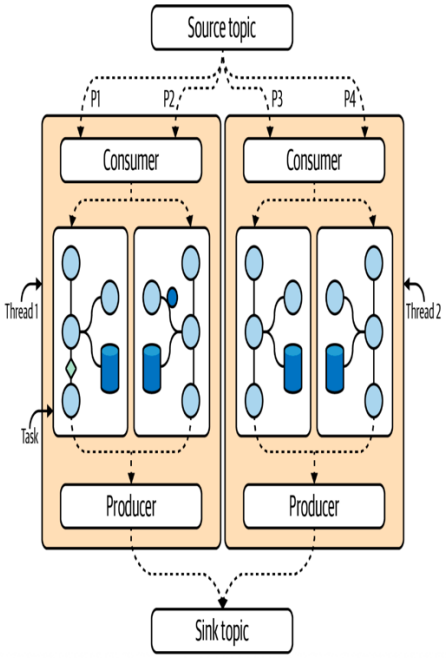
max(source_topic_1_partitions, ..., source_topic_n_partitions)

به‌عنوان مثال، اگر توپولوژی شما از یک موضوع منبع با 16 پارتیشن بخواند، Kafka Streams 16تسک ایجاد می‌کند که هر کدام نسخه‌ای از توپولوژی پردازنده را اجرا می‌کنند. پس از ایجاد تمام تسک‌ها، Kafka Streamsپارتیشن‌های منبع را به هر تسک اختصاص می‌دهد.

تسک‌ها واحدهای منطقی هستند که برای ایجاد و اجرای توپولوژی پردازنده استفاده می‌شوند، در حالی که **رشته‌ها** همان چیزی هستند که تسک را واقعاً اجرا می‌کنند. رشته‌های جریان در Kafka Streams به‌صورت ایزوله و thread-safe طراحی شده‌اند.

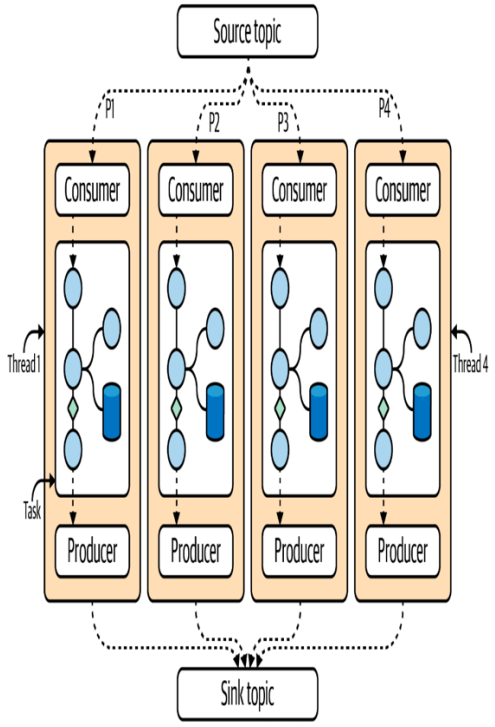
بر خلاف تسک‌ها، Kafka Streamsفرمولی برای تعیین تعداد رشته‌ها ارائه نمی‌دهد. شما مسئول مشخص کردن تعداد رشته‌ها از طریق پراپرتی num.stream.threadsهستید. بیشینه تعداد رشته‌ها برابر با تعداد تسک‌ها است و استراتژی‌های مختلفی برای تصمیم‌گیری درباره تعداد رشته‌های جریان وجود دارد.

برای درک بهتر، تصور کنید برنامه Kafka Streams ما از یک موضوع منبع با چهار پارتیشن (p1 تا p4) می‌خواند. اگر برنامه را با دو رشته (num.stream.threads = 2) اجرا کنیم، چهار تسک ایجاد می‌شود و بین دو رشته توزیع می‌شود، همان‌طور که در شکل **Figure 2-8**نشان داده شده است.



شکل 20

اجرای بیش از یک تسک در هر رشته کاملاً مجاز است، اما گاهی اوقات مطلوب است که تعداد رشته‌ها را افزایش دهیم تا از منابع CPU موجود به‌طور کامل استفاده کنیم. افزایش تعداد رشته‌ها تعداد تسک‌ها را تغییر نمی‌دهد، اما توزیع تسک‌ها بین رشـــته‌ها را تغییر می‌دهد. به‌عنوان مثال، اگر همان برنامه Kafka Streams را طوری پیکربندی کنیم که با چهار رشـــته به‌جای دو رشـــته اجرا شـــود(num.stream.threads = 4) ، توزیع تســـک‌ها و رشـــته‌ها همان‌طور که در شکل **Figure 2-9**نشان داده شده است، خواهد بود.



شکل 21

حالا که با معماری Kafka Streams آشنا شدیم، بیایید بهAPI هایی که Kafka Streams برای ایجاد برنامه‌های پردازش جریان ارائه می‌دهد، نگاهی بیندازیم.

DSLسطح بالا در مقابل API پردازنده سطح پایین

راه‌حل‌های مختلف در لایه‌های متفاوتی از انتزاع ارائه می‌شوند. —جیمز کلیر

یک مفهوم رایج در مهندسی نرم‌افزار این است که انتزاع معمولاً هزینه‌ای دارد: هرچه جزئیات بیشتری را انتزاع کنید، نرم‌افزار بیشتر شبیه به «جادو» به نظر می‌رسد و کنترل بیشتری را از دست می‌دهید. وقتی با Kafka Streams شروع می‌کنید، ممکن است بپرسید با انتخاب استفاده از کتابخانه سطح بالا برای پیاده‌سازی یک برنامه پردازش جریان، چه میزان کنترل را نسبت به استفاده مستقیم ازAPI های سطح پایین Consumer/Producer از دست خواهید داد.

خوشبختانه، Kafka Streamsبه توسعه‌دهندگان اجازه می‌دهد سطح انتزاعی را انتخاب کنند که بهترین کارایی را برای پروژه، تجربه و ترجیحات خودشان داشته باشد.

دو API که می‌توانید از آن‌ها استفاده کنید عبارتند از:

- **DSLسطح بالا**
- **APIپردازنده سطح پایین**

سطح نسبی انتزاع برای هر دو API سطح بالا و سطح پایین در شکل **Figure 2-10**نشان داده شده است.

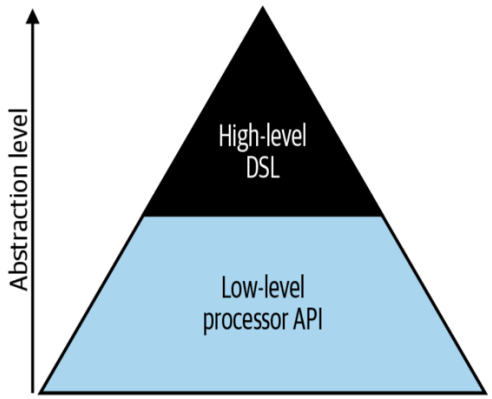


Figure 2-10. Abstraction levels of Kafka Streams APIs

شکل 22

DSLسطح بالا بر روی API پردازنده ساخته شده است، اما رابط کاربری که هرکدام ارائه می‌دهند کمی متفاوت است. اگر می‌خواهید برنامه پردازش جریان خود را با سبک برنامه‌نویسی تابعی بسازید و از انتزاعات سطح بالاتر برای کار با داده‌ها (جریان‌ها و جدول‌ها) استفاده کنید، DSLمناسب شما است.

از سوی دیگر، اگر به دسترسی سطح پایین‌تر به داده‌ها نیاز دارید (مثلاً دسترسی به متاداده رکوردها)، توانایی برنامه‌ریزی توابع دوره‌ای، دسترسی دقیق‌تر به وضعیت برنامه یا کنترل ریزتر بر زمان‌بندی عملیات خاص، APIپردازنده انتخاب بهتری است. در آموزش بعدی، مثال‌هایی از هر دو DSL و API پردازنده خواهید دید و در فصل‌های بعدی به جزئیات بیشتری درباره هر دو می‌پردازیم.

بهترین راه برای دیدن تفاوت بین این دو سطح انتزاع، استفاده از مثال کد است. بیایید به اولین آموزش Kafka Streams خود برویم.**Hello, Streams** :

معرفی آموزشHello, Streams :

در این بخش، تجربه عملی اولیه با Kafka Streams خواهیم داشت. این نسخه‌ای از آموزش استاندارد "Hello, world"است که برای یادگیری زبان‌ها و کتابخانه‌های برنامه‌نویسی جدید استفاده می‌شود. دو پیاده‌سازی برای این آموزش وجود دارد: اولی از DSL سطح بالا و دومی از API پردازنده سطح پایین استفاده می‌کند. هر دو برنامه از نظر عملکردی معادل هستند و هر بار که پیامی از موضوع **users**در Kafka دریافت کنند، یک پیام ساده چاپ می‌کنند (مثلاً با دریافت پیامMitch ، هر برنامه پیام Hello, Mitch را چاپ می‌کند).

راه‌اندازی پروژه

تمام آموزش‌های این کتاب به یک کلاستر Kafka در حال اجرا نیاز دارند و کد منبع هر فصل شامل یک فایل docker-compose.ymlاست که امکان راه‌اندازی یک کلاستر توسعه با Docker را فراهم می‌کند. از آنجایی که برنامه‌های Kafka Streamsقرار است خارج از کلاستر Kafka اجرا شوند (مثلاً روی ماشین‌هایی غیر از بروکرها)، بهتر است کلاستر Kafka را به عنوان یک بخش زیرساخت جداگانه در نظر بگیریم که مورد نیاز است اما با برنامه Kafka Streamsشما متفاوت است.

برای شروع اجرای کلاسترKafka ، مخزن را کلون کرده و به دایرکتوری آموزش این فصل بروید. دستورات زیر این کار را انجام می‌دهند:

```
$ git clone git@github.com:mitch-seymour/mastering-kafka-streams-and-ksqldb.git

$ cd mastering-kafka-streams-and-ksqldb/chapter-02/hello-streams
```

سپس، با اجرای دستور زیر، کلاستر Kafka را راه‌اندازی کنید:

```
docker-compose up
```

بروکر بر روی پورت 29092 گوش خواهد داد. علاوه بر این، دستور فوق یک کانتینر را اجرا می‌کند که موضوع **users** مورد نیاز برای این آموزش را از قبل ایجاد می‌کند. حالا که کلاستر Kafka ما در حال اجرا است، می‌توانیم شروع به ساخت برنامه Kafka Streams خود کنیم.

ایجاد پروژه جدید

در این کتاب، از ابزار ساختی به نام **Gradle** برای کامپایل و اجرای برنامه‌های Kafka Streams خود استفاده خواهیم کرد. ابزارهای ساخت دیگر (مثل Maven) نیز پشتیبانی می‌شوند، اما ما Gradle را به دلیل خوانایی بهتر فایل‌های ساخت آن انتخاب کرده‌ایم.

علاوه بر توانایی کامپایل و اجرای کد، Gradle می‌تواند برای راه‌اندازی سریع برنامه‌های Kafka Streams جدیدی که خارج از این کتاب می‌سازید نیز استفاده شود. این کار با ایجاد یک دایرکتوری برای پروژه و سپس اجرای دستور **gradle init** از داخل آن دایرکتوری انجام می‌شود. یک مثال از این روند به صورت زیر است:

```
$ mkdir my-project && cd my-project
$ gradle init \
--type java-application \
--dsl groovy \
--test-framework junit-jupiter \
--project-name my-project \
--package com.example
```

کد منبع این کتاب از قبل ساختار پروژه‌ی اولیه برای هر آموزش را دارد، بنابراین اجرای **gradle init** ضروری نیست مگر اینکه بخواهید پروژه‌ی جدیدی برای خودتان بسازید. ما صرفاً به این موضوع اشاره می‌کنیم با این فرض که در آینده قصد دارید برنامه‌های Kafka Streams خودتان را بنویسید و به دنبال راه سریع برای راه‌اندازی پروژه بعدی هستید.

در اینجا ساختار پایه‌ای یک پروژه Kafka Streams آورده شده است:

```
.
├── build.gradle
└── src
├── main
│   ├── java
│   └── resources
└── test
└── java
```

این فایل **build** پروژه است. در آن تمام وابستگی‌های مورد نیاز برنامه ما (از جمله کتابخانه Kafka Streams) مشخص می‌شوند.

کد منبع و تعریف‌های توپولوژی را در مسیر **src/main/java** ذخیره می‌کنیم. مسیر **src/main/resources** معمولاً برای نگهداری فایل‌های پیکربندی استفاده می‌شود.

تســـــت‌هــای واحــد و توپولوژی، کــه در بخش «Testing Kafka Streams» بــه آن‌هــا پرداختــه می‌شـــود، در مسیر **src/test/java**قرار خواهند گرفت.

حال که یاد گرفتیم چگونه پروژه‌های Kafka Streams را راه‌اندازی کنیم و نگاهی اولیه به ساختار پروژه داشتیم، بیایید ببینیم چگونه Kafka Streams را به پروژه اضافه کنیم.

افزودن وابستگی Kafka Streams

برای شــروع کار باKafka Streams ، کافی اســت کتابخانه Kafka Streams را به عنوان وابســتگی در فایل build پروژه اضـــافه کنیم. (در پروژه‌هایGradle ، فایل build ما **build.gradle**نام دارد.) نمونه‌ای از این فایل build در ادامه آمده است:

```
plugins {
    id 'java'
    id 'application'
}

repositories {
    jcenter()
}

dependencies {
    implementation 'org.apache.kafka:kafka-streams:2.7.0' ❶
}

task runDSL(type: JavaExec) { ❷
    main = 'com.example.DslExample'
    classpath sourceSets.main.runtimeClasspath
}

task runProcessorAPI(type: JavaExec) { ❸
    main = 'com.example.ProcessorApiExample'
    classpath sourceSets.main.runtimeClasspath
}
```

کتابخانه Kafka Streams را به پروژه خود اضافه کنید. این آموزش در میان سایر آموزش‌های این کتاب منحصر به فرد است، زیرا ما دو نسخه متفاوت از توپولوژی خود ایجاد خواهیم کرد. این خط یک **کار Gradle** برای اجرای نسخهDSL برنامه ما اضـافه می‌کند. به همین ترتیب، این خط یک **کار Gradle** برای اجرای نسـخه Processor API برنامه اضـافه می‌کند.

برای ســاخت پروژه خود (که در واقع وابســتگی را از مخزن راه دور به پروژه شــما می‌آورد)، می‌توانیم دســتور زیر را اجرا کنیم:

```
./gradlew build
```

همین Kafka Streams !نصب شد و آماده استفاده است. حالا بیایید ادامه آموزش را دنبال کنیم.

DSL

مثال DSL بســیار ســاده اســت. ابتدا نیاز داریم از کلاســی به نام **StreamsBuilder**برای ســاخت توپولوژی پردازنده استفاده کنیم:

```
StreamsBuilder builder = new StreamsBuilder();
```

سپس، همان‌طور که در بخش «Processor Topologies» یاد گرفتیم، نیاز داریم یک **پردازنده منبع (source processor)** اضافه کنیم تا داده‌ها را از یک موضوع Kafka بخواند (در این مثال، موضوع ما **users** خواهد بود). چند روش مختلف برای این کار وجود دارد که بسته به نحوه مدل‌سازی داده‌هایمان انتخاب می‌کنیم، اما فعلاً داده‌ها را به صورت یک جریان مدل‌سازی می‌کنیم. خط زیر پردازنده منبع را اضافه می‌کند:

```
KStream<Void, String> stream = builder.stream("users");
```

در فصل بعدی بیشتر توضیح خواهیم داد، اما نوع‌های جنریک در KStream<Void, String> به نوع کلید و مقدار اشاره دارند. در این مثال، کلید خالی است (Void) و مقدار از نوع String است.

حالا نوبت اضافه کردن یک **پردازنده جریان (stream processor)** است. از آنجا که ما فقط می‌خواهیم یک پیام خوش‌آمدگویی ساده برای هر پیام چاپ کنیم، می‌توانیم از عملگر **foreach** با یک lambda ساده استفاده کنیم.

```
stream.foreach(
(key, value) -> {
System.out.println("(DSL) Hello,
" + value);
});
```

در نهایت، زمان آن رسیده که توپولوژی خود را بسازیم و برنامه پردازش جریان خود را اجرا کنیم:

```
KafkaStreams streams = new KafkaStreams(builder.build(), config);

streams.start();
```

کد کامل، شامل مقداری کد پیش‌نیاز برای اجرای برنامه، در **Example 2-1** نشان داده شده است.

Example 2-1. Hello, world—DSL example

```
class DslExample {

  public static void main(String[] args) {
    StreamsBuilder builder = new StreamsBuilder(); ❶

    KStream<Void, String> stream = builder.stream("users"); ❷

    stream.foreach( ❸
        (key, value) -> {
          System.out.println("(DSL) Hello, " + value);
        });

    // omitted for brevity
    Properties config = ...; ❹

    KafkaStreams streams = new KafkaStreams(builder.build(), config); ❺
    streams.start();

    // close Kafka Streams when the JVM shuts down (e.g., SIGTERM)
    Runtime.getRuntime().addShutdownHook(new Thread(streams::close)); ❻
  }
}
```

- از **builder** برای ساخت توپولوژی استفاده می‌کنیم.
- یک **source processor** اضافه کنید که داده‌ها را از موضوع **users** بخواند.

- از اپراتور **foreach** در DSL برای چاپ یک پیام ساده استفاده کنید .شامل اپراتورهای زیادی است که در فصل‌های بعدی بررسی خواهیم کرد.
- پیکربندی Kafka Streams برای اختصار حذف شده است، اما در فصل‌های بعدی توضیح داده خواهد شد. این پیکربندی مشخص می‌کند که برنامه باید از کدام کلاستر Kafka داده بخواند و عضو کدام **consumer group** است.
- توپولوژی را بسازید و جریان داده‌ها را شروع کنید.
- وقتی JVM خاموش می‌شود، Kafka Streams را ببندید.

برای اجرای برنامه، دستور زیر را اجرا کنید:

```
./gradlew runDSL --info
```

اکنون برنامه Kafka Streams شما در حال اجراست و به داده‌های ورودی گوش می‌دهد. همان‌طور که در بخش «Hello, Kafka» دیدیم، می‌توانیم با استفاده از اسکریپت **kafka-console-producer** داده‌ای به کلاستر Kafka تولید کنیم.

```
docker-compose exec kafka bash
kafka-console-producer \
--bootstrap-server localhost:9092 \
--topic users
```

اسکریپت‌های کنسول در کانتینر Kafka موجود هستند که بروکر را در کلاستر توسعه ما اجرا می‌کند. همچنین می‌توانید این اسکریپت‌ها را به‌عنوان بخشی از توزیع رسمی Kafka دانلود کنید.

یک **producer محلی** راه‌اندازی کنید که داده‌ها را به موضوع **users** بنویسد. هنگامی که در **prompt** مربوط به producer هستید، با تایپ نام کاربر و زدن **Enter**، یک یا چند رکورد ایجاد کنید. وقتی کارتان تمام شد، با **Control-C** از prompt خارج شوید:

```
>angie

>guy

>kate

>mark
```

برنامه Kafka Streams شما باید پیام‌های خوش‌آمدگویی زیر را تولید کند.

```
(DSL) Hello, angie
(DSL) Hello, guy
(DSL) Hello, kate
(DSL) Hello, mark
```

اکنون تأیید کرده‌ایم که برنامهٔ ما همان‌طور که انتظار می‌رود کار می‌کند. در چند فصل آینده چند مورد استفادهٔ جالب‌تر را بررسی خواهیم کرد، اما این فرایندِ تعریف یک توپولوژی و اجرای برنامه، پایه‌ای است که می‌توانیم روی آن ادامه دهیم. حالا بیایید ببینیم چگونه می‌توان همین توپولوژی Kafka Streams را با استفاده از **Processor API** در سطح پایین‌تر ایجاد کرد.

Processor API

Processor APIبرخی از انتزاع‌هایی را که در DSL سطح بالا وجود دارند در اختیار ندارد و نحو آن بیشتر به‌طور مستقیم یادآوری می‌کند که ما در حال ساخت توپولوژی‌های پردازنده هستیم؛ با متدهایی مانند Topology.addSource، Topology.addProcessor و Topology.addSink(که آخری در این مثال استفاده نشده است).

اولین گام برای استفاده از توپولوژی پردازنده این است که یک نمونهٔ جدید از Topologyبسازیم، به شکل زیر:

```
Topology topology = new Topology();
```

در ادامه یک پردازندهٔ منبع (source processor) برای خواندن داده از تاپیک usersایجاد می‌کنیم و یک پردازندهٔ جریان (stream processor) برای چاپ یک پیام خوشامد ساده می‌سازیم. پردازندهٔ جریان به کلاسی به نام SayHelloProcessorارجاع می‌دهد که کمی بعد آن را پیاده‌سازی خواهیم کرد:

اولین آرگومان متد addSourceیک نام دلخواه برای این پردازندهٔ جریان است. در اینجا ما این پردازنده را به سادگی **UserSource**می‌نامیم. در خط بعدی، زمانی که می‌خواهیم یک پردازندهٔ فرزند را متصل کنیم، به این نام ارجاع می‌دهیم؛ این کار مشخص می‌کند که داده‌ها چگونه در توپولوژی ما جریان پیدا کنند. آرگومان دوم نام تاپیکی است که این پردازندهٔ منبع باید از آن داده بخواند (در اینجا users).

این خط یک پردازندهٔ پایین‌دستی (downstream) جدید به نام **SayHello**ایجاد می‌کند که منطق پردازشی آن در کلاس SayHelloProcessorتعریف شده است (در بخش بعدی آن را ایجاد خواهیم کرد). در **Processor API**می‌توانیم یک پردازنده را با مشخص کردن نام پردازندهٔ والد به پردازندهٔ دیگری متصل کنیم. در اینجا پردازندهٔ **UserSource**را به‌عنوان والد پردازندهٔ **SayHello**مشخص می‌کنیم؛ یعنی داده‌ها از **UserSource**به **SayHello**جریان پیدا می‌کنند.

همان‌طور که قبلاً در آموزش DSL دیدیم، اکنون باید توپولوژی را بسازیم و برای اجرای آن streams.start()را فراخوانی کنیم:

```
KafkaStreams streams = new KafkaStreams(topology, config);

streams.start();
```

قبل از اجرای کد، باید کلاس SayHelloProcessorرا پیاده‌سازی کنیم. هر زمان که با استفاده از **Processor API**یک پردازندهٔ جریان سفارشی می‌سازید، لازم است رابط Processorرا پیاده‌سازی کنید. این رابط متدهایی را برای مقداردهی اولیهٔ پردازندهٔ جریان(init) ، اعمال منطق پردازش جریان روی یک رکورد منفرد (process) و یک تابع پاکسازی (close) مشخص می‌کند. در این مثال به توابع مقداردهی اولیه و پاکسازی نیازی نداریم.

در ادامه یک پیاده‌سازی ساده از SayHelloProcessorآورده شده که در این مثال از آن استفاده خواهیم کرد. در فصل ۷ مثال‌های پیچیده‌تر و همچنین همهٔ متدهای رابط Processor (یعنی init، process و close)را با جزئیات بیشتری بررسی خواهیم کرد.

```
public class SayHelloProcessor implements Processor<Void, String,
Void, Void> {
@Override
public void init(ProcessorContext<Void, Void> context) {}
@Override
public void process(Record<Void, String> record) {
System.out.println("(Processor API) Hello,
" + record.value());
}
@Override
public void close() {}
}
```

دو جنریک اول در رابط Processor) در این مثال Processor<Void, String, ..., ...>) به نوع **کلید ورودی** و **مقدار ورودی** اشاره دارند. از آنجا که کلیدهای ما null هستند و مقادیر ما **نام کاربر**) یعنی رشته‌های متنی) هستند، استفاده از Void و String انتخاب مناسبی است. دو جنریک آخر (Processor<..., ..., Void, Void>) به نوع **کلید خروجی** و **مقدار خروجی** اشاره دارند. در این مثال، SayHelloProcessor فقط یک پیام خوشامد چاپ می‌کند. از آنجا که هیچ کلید یا مقدار خروجی‌ای را به پردازنده‌های پایین‌دستی ارسال نمی‌کنیم، Void نوع مناسبی برای دو جنریک آخر است.

در این مثال به مقداردهی اولیهٔ خاصی نیاز نداریم، بنابراین بدنهٔ این متد خالی است. جنریک‌های رابط (ProcessorContext یعنی ProcessorContext<Void, Void>) به نوع کلید و مقدار خروجی اشاره دارند (باز هم چون در این مثال هیچ پیامی را به پردازنده‌های پایین‌دستی ارسال نمی‌کنیم، هر دو Void هستند.)

- منطق پردازش در متدی با نام مناسب process در رابط Processor قرار دارد. در اینجا ما یک پیام خوشامد ساده چاپ می‌کنیم. توجه داشته باشید که جنریک‌های رابط Record به نوع کلید و مقدار **رکوردهای ورودی** اشاره دارند.

در این مثال به عملیات پاکسازی خاصی هم نیازی نیست.

اکنون می‌توانیم کد را با همان دستوری اجرا کنیم که در مثال DSL استفاده کردیم:

```
./gradlew runProcessorAPI --info
```

باید خروجی زیر را مشاهده کنید که نشان می‌دهد برنامهٔ **Kafka Streams** شما همان‌طور که انتظار می‌رود در حال کار است:

```
(Processor API) Hello, angie
(Processor API) Hello, guy
(Processor API) Hello, kate
(Processor API) Hello, mark
```

با وجود قدرت زیاد **Processor API** که در فصل ۷ خواهیم دید، در بسیاری از موارد استفاده از **DSL** ترجیح داده می‌شود، زیرا علاوه بر مزایای دیگر، شامل دو انتزاع بسیار قدرتمند است **stream** :و **table.** در بخش بعدی اولین نگاه خود را به این انتزاع‌ها خواهیم داشت.

Streams و Tables

اگر با دقت به مثال ۱-۲ نگاه کنید، متوجه می‌شوید که از عملگر DSL به نام stream برای خواندن یک تاپیک Kafka در قالب یک **stream** استفاده کرده‌ایم. خط کد مربوطه به شکل زیر است:

```
KStream<Void, String> stream = builder.stream("users");
```

با این حال، **Kafka Streams** یک روش دیگر برای مشاهدهٔ داده‌های ما نیز پشتیبانی می‌کند: به‌صورت **table.** در این بخش هر دو گزینه را بررسی می‌کنیم و یاد می‌گیریم چه زمانی باید از **stream** و چه زمانی از **table** استفاده کنیم.

همان‌طور که در بخش «Processor Topologies» توضیح داده شد، طراحی یک توپولوژی پردازنده شامل مشخص کردن مجموعه‌ای از **پردازنده‌های منبع (source)** و **پردازنده‌های مقصد (sink)** است که به تاپیک‌هایی مربوط می‌شوند که برنامهٔ شما از آن‌ها می‌خواند یا در آن‌ها می‌نویسد. با این حال، به‌جای کار مستقیم با تاپیک‌های Kafka ، **Kafka**

Streams DSLبه شـــما اجازه می‌دهد با نمایش‌های مختلفی از یک تاپیک کار کنید که هرکدام برای موارد اســـتفادهٔ متفاوتی مناسب هستند.

دو روش برای مدل‌سازی داده‌ها در تاپیک‌های Kafka وجود دارد:

- به‌صورت) **stream**که به آن **record stream**نیز گفته می‌شود(
- به‌صورت) **table**که به آن **changelog stream**نیز گفته می‌شود(

ساده‌ترین راه برای مقایسهٔ این دو مدل داده استفاده از یک مثال است.

فرض کنید تاپیکی داریم که شامل **لاگ‌های SSH** است و هر رکورد آن با **شناسهٔ کاربر (user ID)** کلیدگذاری شده است، همان‌طور که در جدول ۲-۲ نشان داده شده است.

Table 2-2. Keyed records in a single topic-partition

Key	Value	Offset
mitch	`{ "action": "login" }`	0
mitch	`{ "action": "logout" }`	1
elyse	`{ "action": "login" }`	2
isabelle	`{ "action": "login" }`	3

جدول 2

قبل از مصرف این داده‌ها، باید تصمیم بگیریم از کدام انتزاع استفاده کنیم **stream** :یا **table**. هنگام گرفتن این تصمیم، باید در نظر بگیریم که آیا می‌خواهیم فقط **آخرین وضعیت/نمایش یک کلید مشخص** را نگه داریم یا **کل تاریخچهٔ پیام‌ها** را.

بیایید این دو گزینه را در کنار هم مقایسه کنیم.

Streams

می‌توان آن‌ها را در اصـــطلاح پایگاه‌داده شـــبیه **عملیات insert** در نظر گرفت. هر رکورد مجزا در این نمای لاگ باقی می‌ماند. نمایش stream از تاپیک ما را می‌توان در جدول ۳-۲ مشاهده کرد.

Table 2-3. Stream view of ssh logs

Key	Value	Offset
mitch	{ "action": "login" }	0
mitch	{ "action": "logout" }	1
elyse	{ "action": "login" }	2
isabelle	{ "action": "login" }	3

جدول 3

Tables

می‌توان آن‌ها را مشـــابه **عملیات update در پایگاه‌داده** در نظر گرفت. در این نمای لاگ‌ها، فقط **وضـــعیت فعلی**)یعنی آخرین رکورد برای یـک کلیـد مشـــخص یـا نوعی تجمیع از آن) برای هر کلیـد نـگـه داشـــتـه می‌شـــود. Tableهـا مـعـمـولاً از **تـاپـیـک‌هـای compacted** ســـاخـتـه مـی‌شـــونـد (یـعـنـی تـاپـیـک‌هـایـی کـه با cleanup.policy=compactپیکربندی شـــده‌اند؛ این تنظیم به Kafka می‌گوید که فقط **آخرین نمایش هر کلید** نگه داشته شود).

نمای **table**از تاپیک ما در جدول ۴-۲ نشان داده شده است.

Table 2-4. Table view of ssh logs

Key	Value	Offset
mitch	{ "action": "logout" }	1
elyse	{ "action": "login" }	2
isabelle	{ "action": "login" }	3

جدول 4

Tableها ذاتاً **stateful**هستند و اغلب برای انجام **تجمیع‌ها (aggregation)** در Kafka Streams استفاده می‌شوند. در جدول ۴-۲ ما در واقع یک تجمیع ریاضی انجام ندادیم؛ فقط **آخرین رویداد ssh برای هر شناسهٔ کاربر** را نگه داشتیم. با این حال، tableها از **تجمیع‌های ریاضی** نیز پشتیبانی می‌کنند.

برای مثال، به‌جای دنبال کردن آخرین رکورد برای هر کلید، می‌توانستیم به‌راحتی **یک شمارش تجمعی (rolling count)** محاسبه کنیم. در این صورت به یک table کمی متفاوت می‌رسیدیم که در آن مقادیر شامل **نتیجهٔ تجمیع شمارش** ما هستند. می‌توانید یک table با تجمیع شمارش را در جدول ۵-۲ مشاهده کنید.

Table 2-5. Aggregated table view of ssh logs

Key	Value	Offset
mitch	2	1
elyse	1	2
isabelle	1	3

جدول 5

خوانندگان دقیق ممکن است متوجه یک ناهماهنگی بین **طراحی لایهٔ ذخیره‌سازی (Kafka** که یک لاگ توزیع‌شده و فقط‌الحاقی *append-only*است) و مفهوم **table**شده باشند. رکوردهایی که در Kafka نوشته می‌شوند **تغییرناپذیر (immutable)**هستند، بنابراین چگونه ممکن است داده‌ها را به شکل **به‌روزرسانی (update)** و در قالب نمایش tableاز یک تاپیک Kafka مدل‌سازی کرد؟

پاسخ ساده است **table :در سمت Kafka Streams ساخته (materialized) می‌شود** و برای این کار از یک **key-value store**استفاده می‌کند که به‌طور پیش‌فرض با **RocksDB**پیاده‌سازی شده است.

با مصرف یک **جریان مرتب از رویدادها** و نگه داشتن **فقط آخرین رکورد برای هر کلید** در key-value store سمت کلاینت (که در اصطلاح Kafka Streams معمولاً **state store**نامیده می‌شود)، در نهایت به یک نمایش **شبیه table یا map** از داده‌ها می‌رسیم. به بیان دیگر، **tableچیزی نیست که مستقیماً از Kafka بخوانیم**، بلکه چیزی است که **در سمت کلاینت می‌سازیم**.

در واقع می‌توانید با چند خط کد جاوا این ایدهٔ ساده را پیاده‌سازی کنید. در قطعه‌کد زیر، Listنمایانگر یک **stream**است، زیرا شامل مجموعه‌ای مرتب از رکوردهاست، و **table**با پیمایش این لیست (stream.forEach) ساخته می‌شود؛ به این صورت که با استفاده از یک Mapفقط **آخرین رکورد برای هر کلید** نگه داشته می‌شود. کد جاوای زیر این ایدهٔ پایه را نشان می‌دهد:

```
import java.util.Map.Entry;
var stream = List.of(
Map.entry("a"
Map.entry("b"
```

```
Map.entry("a"
, 1),
, 1),
, 2));
var table = new HashMap<>();
stream.forEach((record) -> table.put(record.getKey(),
record.getValue()));
```

اگر پس از اجرای این کد، **stream**و **table**را چاپ کنید، خروجی زیر را مشاهده خواهید کرد:

```
stream ==> [a=1, b=1, a=2]
table ==> {a=2, b=1}
```

البته، پیاده‌سازی **Kafka Streams**از این مفهوم پیشرفته‌تر است و می‌تواند از ساختارهای داده‌ای مقاوم در برابر خطا استفاده کند، نه فقط یک Mapدر حافظهٔ اصلی. اما این توانایی برای ساخت یک **نمای table** از یک **جریان نامحدود** تنها یک سوی رابطهٔ پیچیده‌تر بین **stream**و **table**است، که در ادامه بررسی خواهیم کرد.

Stream/Table Duality

دوگانگی **table**و **stream**از این واقعیت ناشی می‌شود که می‌توان **tableها را به شکل stream نمایش داد** و **streamها را برای بازسازی table ها استفاده کرد** .ما تبدیل دوم، یعنی تبدیل یک stream به table را در بخش قبلی دیدیم، زمانی که دربارهٔ تفاوت بین **لاگ فقطالحاقی و تغییرناپذیر Kafka** و مفهوم یک **ساختار table تغییرپذیر** که می‌تواند داده‌ها را به‌روزرسانی کند، صحبت کردیم.

این توانایی برای بازسازی table از stream تنها مخصوص Kafka Streams نیست و در انواع مختلف ذخیره‌سازی داده‌ها نسبتاً رایج است.
برای مثال، فرایند **replicationدر MySQL** بر اساس همین مفهوم کار می‌کند: با استفاده از یک جریان از رویدادها (یعنی تغییرات ردیف‌ها) می‌توان **جدول منبع را در یک کپی پایین‌دستی بازسازی کرد** .به همین ترتیب، در **Redis**یک فایل **append-only (AOF)**وجود دارد که تمام دستورات نوشته‌شده به حافظهٔ key-value را ذخیره می‌کند. اگر سرور Redis آفلاین شود، می‌توان با بازپخش جریان دستورات درAOF ، **داده‌ها را بازسازی کرد.**

و سوی دیگر (نمایش table به‌صورت stream) چیست؟

وقتی یک table را مشاهده می‌کنید، در واقع **نمایی نقطه‌ای از یک stream** را می‌بینید. همان‌طور که قبلاً دیدیم، tableها می‌توانند با رسیدن رکورد جدید به‌روزرسانی شوند. با تغییر دید ما از table به‌stream ، می‌توانیم به‌سادگی **به‌روزرسانی را به شکل یک insert پردازش کنیم** و رکورد جدید را به انتهای لاگ اضافه کنیم، به جای اینکه کلید را به‌روزرسانی کنیم.

این شهود را می‌توان با چند خط کد جاوا به‌خوبی نشان داد:

```
var stream = table.entrySet().stream().collect(Collectors.toList());
stream.add(Map.entry("a"
, 3));
```

این بار، اگر محتوای **stream**را چاپ کنید، خواهید دید که دیگر از **معنای به‌روزرسانی (update semantics)**استفاده نمی‌کنیم، بلکه از **معنای درج (insert semantics)** استفاده شده است:

```
stream ==> [a=2, b=1, a=3]
```

تاکنون با کتابخانه‌های استاندارد جاوا کار کرده‌ایم تا درک بهتری از **stream**و **table**پیدا کنیم. با این حال، هنگام کار با stream و table در **Kafka Streams**از مجموعه‌ای از انتزاع‌های تخصصی‌تر استفاده خواهید کرد. در ادامه این انتزاع‌ها را بررسی می‌کنیم.

KStream, KTable, GlobalKTable

یکی از مزایای استفاده از **DSLسطح بالا** نسبت به **Processor APIسطح پایین** در Kafka Streams این است که DSLسطح بالا شامل مجموعه‌ای از انتزاع‌هاست که کار با stream و table را بسیار آسان می‌کند.

مروری کوتاه بر هر کدام:

KStream

یک KStream نمایی از یک **رکورد جریان‌بندی‌شده (partitioned record stream)** است که در آن داده‌ها با **معنای درج (insert semantics)** نمایش داده می‌شوند (یعنی هر رویداد مستقل از سایر رویدادها در نظر گرفته می‌شود.(

KTable

یک KTable نمایی از یک **tableبخش‌بندی‌شده (partitioned table) یا (changelog stream)** است که در آن داده‌ها با **معنای به‌روزرسانی (update semantics)** نمایش داده می‌شوند (آخرین وضعیت یک کلید مشخص توسط برنامه دنبال می‌شود). از آنجا کهKTable ها بخش‌بندی‌شده هستند، هر تسک Kafka Streams فقط یک زیرمجموعه از جدول کامل را دارد.

GlobalKTable

این مشابه KTable است، با این تفاوت که هر GlobalKTable شامل یک نسخه کامل (یعنی غیر بخش‌بندی‌شده) از داده‌های پایه است. در فصل ۴ خواهیم آموخت که چه زمانی باید از KTable و چه زمانی از GlobalKTable استفاده کرد.

برنامه‌های Kafka Streams می‌توانند از چند انتزاع stream/table همزمان استفاده کنند یا فقط یکی از آن‌ها را به کار گیرند. این کاملاً به نیازمندی‌های شما بستگی دارد و در فصول بعدی خواهید آموخت که چه زمانی از هر کدام استفاده کنید.

جمع‌بندی

تبریک! شما اولین تجربهٔ خود با **Kafka Streams**را پشت سر گذاشتید. آنچه یاد گرفتید:

- Kafka Streamsدر لایهٔ **پردازش جریان** اکوسیستم Kafka قرار دارد. اینجا جایی است که پردازش، تبدیل و غنی‌سازی داده‌ها اتفاق می‌افتد.
- Kafka Streamsبرای ساده کردن توسعهٔ برنامه‌های پردازش جریان طراحی شده است؛ با یک API ساده و تابعی و مجموعه‌ای از **اصول پردازش جریان** که می‌توانند در پروژه‌های مختلف دوباره استفاده شوند. وقتی نیاز به کنترل بیشتر باشد، می‌توان از **Processor APIسطح پایین** برای تعریف توپولوژی استفاده کرد.
- Kafka Streamsیادگیری ساده‌تری دارد و مدل استقرار آن نسبت به راه‌حل‌های مبتنی بر کلاستر مثل **Apache Flink**و **Apache Spark Streaming**ساده‌تر است. همچنین از پردازش **رویداد به رویداد** پشتیبانی می‌کند که به عنوان **true streaming**شناخته می‌شود.

- Kafka Streamsبرای حل مسائلی که نیازمند تصمیم‌گیری و پردازش داده به‌صورت **real-time**هستند عالی است. همچنین **قابل اعتماد، نگهداری‌پذیر، مقیاس‌پذیر و انعطاف‌پذیر** است.
- نصب و اجرای Kafka Streams ساده است و مثال‌های کد این فصل در GitHub
قابل دسترسی هستند.

در فصل بعد، با **پردازش بدون حالت (stateless processing)** در Kafka Streams آشنا می‌شویم و تجربه عملی با چند **عملگر جدیدDSL** به دست می‌آوریم تا برنامه‌های پردازش جریان پیشرفته و قدرتمندتری بسازیم.

نکات و یادداشت ها

۱ .منظور ما از **اکوسیستم رسمی**، تمام مؤلفه‌هایی است که تحت پروژهٔ **Apache Kafka**نگهداری می‌شوند.
۲**Jay Kreps** .، یکی از نویسندگان اصلیApache Kafka ، در سال ۲۰۱۴ این موضوع را به‌طور مفصل در یک پست وبلاگی در O'Reilly بررسی کرده است.
۳ .این شامل **streams/tableهای تجمیع‌شده** نیز می‌شود که بعداً در همین فصل بررسی خواهیم کرد.
۴ .ما یک فصل کامل به **زمان** اختصاص داده‌ایم؛ همچنین می‌توانید ارائهٔ Matthias J. Sax از Kafka Summit 2019 را ببینید.
۵**Guozhang Wang** .، که نقش کلیدی در توسعهٔ Kafka Streams داشت، بیشتر شایستهٔ قدردانی است چرا که KIP اصلی برای Kafka Streams را ارائه کرد .لینک
۶ Kafka Connect .کمی وارد حوزهٔ پردازش رویداد شد با اضافه کردن پشتیبانی از **single message transforms**، اما این قابلیت بسیار محدودتر از Kafka Streams است.
۷ Kafka Streams .با زبان‌های JVM مانند Scala و Kotlin نیز کار می‌کند، اما در این کتاب صرفاً از جاوا استفاده شده است.
۸ .چند گروه مصرف‌کننده می‌توانند از یک تاپیک استفاده کنند و هر گروه مستقل از گروه‌های دیگر پیام‌ها را پردازش می‌کند.
۹ .در حالی که می‌توان به یک تاپیک موجود پارتیشن اضافه کرد، الگوی پیشنهادی این است که **یک تاپیک جدید با تعداد پارتیشن موردنظر ایجاد کرده** و تمام بارهای کاری موجود را به آن منتقل کنید.
۱۰ .شامل برخی قابلیت‌های خاص برنامه‌های **stateful**نیز می‌شود که در فصل ۴ بررسی خواهد شد.
۱۱ .با توجه به رشد سریع حوزهٔ پردازش جریان، مقایسهٔ تمام راه‌حل‌ها با Kafka Streams دشوار است؛ بنابراین تمرکز ما روی محبوب‌ترین و بالغ‌ترین راه‌حل‌ها در زمان نگارش است.
۱۲ .پیشنهادی برای پشتیبانی از پردازش batch در Kafka Streams وجود دارد، اگرچه قدیمی است.
۱۳ .شامل یک پایگاه دادهٔ رابطه‌ای مبتنی بر Kafka به نامKarelDB ، کتابخانهٔ تحلیل گرافی بر پایهٔ Kafka Streams و غیره .لینک
۱۴ .در زمان نگارش، Apache Flinkبه تازگی نسخهٔ بتای **queryable state**را منتشر کرده بود، اما API هنوز کمتر بالغ بود و هشدار داده شده بود که احتمال تغییرات ناسازگار در نسخه‌های بعدی وجود دارد. بنابراین Kafka Streamsهنوز API بالغ‌تر و آمادهٔ تولید برای **querying state**دارد.
۱۵ .استثنا زمانی است که تاپیک‌ها join می‌شوند؛ در این حالت یک توپولوژی واحد همهٔ تاپیک‌های منبع در join را می‌خواند بدون تقسیم بیشتر به زیر-توپولوژی‌ها.
۱۶ .در این مثال، داده‌ها در دو تاپیک واسطه (valid-mentions) و (invalid-mentions نوشته و بلافاصله مصرف

می‌شوند.

۱۷ .استفاده از تایپک‌های واسطه معمولاً فقط برای برخی عملیات (برای مثال، repartitioning داده) ضروری است.

۱۸ .ما این کار را در اینجا فقط برای اهداف بحث انجام می‌دهیم.

۱۹ .یک برنامهٔ جاوا ممکن است انواع مختلفی از thread ها را اجرا کند.

۲۰ .بحث ما صرفاً روی **stream threads** است که توسط کتابخانهٔ Kafka Streams برای اجرای توپولوژی پردازنده ایجاد و مدیریت می‌شوند.

۲۱ .به یاد داشته باشید، یک توپولوژی Kafka Streams می‌تواند از چند زیر-توپولوژی تشکیل شده باشد، بنابراین برای به دست آوردن تعداد تسک‌های کل برنامه، باید تعداد تسک‌ها را در تمام زیر-توپولوژی‌ها جمع کنید.

۲۲ .این بدان معنا نیست که یک پردازندهٔ جریان ضعیف از مشکلات همزمانی مصون است، اما به‌طور پیش‌فرض **stream threads هیچ state مشترکی ندارند.**

۲۳ .به‌عنوان مثال، تعداد هسته‌هایی که برنامه به آن دسترسی دارد می‌تواند تعداد thread هایی را که تصمیم می‌گیرید اجرا کنید، تعیین کند.

۲۴ .اگر نمونه برنامه روی یک ماشین ۴ هسته‌ای اجرا شود و توپولوژی ۱۶ تسک داشته باشد، ممکن است بخواهید تعداد thread ها را ۴ تنظیم کنید که به شما یک thread برای هر هسته می‌دهد.

۲۵ .از سوی دیگر، اگر همان برنامه ۱۶ تسک روی یک ماشین ۴۸ هسته‌ای اجرا شود، ممکن است بخواهید با ۱۶ thread اجرا کنید (با ۴۸ اجرا نمی‌کنید چون حد بالایی تعداد تسک‌ها است، یا در این مورد: ۱۶.)

۲۶ .از اصول اولیه Elon Musk: دربارهٔ قدرت **تفکر مستقل.**

۲۷ .اگر بخواهید بررسی کنید و telnet نصب داشته باشید، می‌توانید دستور echo 'exit' | telnet localhost 29092 را اجرا کنید. اگر پورت باز باشد، باید پیام "Connected to localhost" را ببینید.

۲۸ .دستورالعمل نصب Gradle در gradle.org موجود است؛ ما نسخهٔ ۶.۶.۱ را برای آموزش‌های این کتاب استفاده کردیم.